```
    for competitor in @competitors
      if toInsert.lastName > competitor.lastName
        insertAt++
    @competitors.splice insertAt, 0, toInsert
```

Wait a minute! What happens when Joe asks you to show only competitors with names starting with *J* or those with competitor numbers starting with 02? Remember that the data is really an array. It's being loaded asynchronously, but it's still an array. If it were an array of data, you wouldn't write a big `for` loop with a bunch of conditionals. Instead, you'd use array methods such as `filter`. What would that look like?

```
fasterThan = (n) ->
  (z) ->
    z.time < n

lastNameStartsWith = (letter) ->
  (s) -> competitor.lastName[0] is letter

result = \
competitors
.filter(fasterThan '12:00')
.filter(lastNameStartsWith 'a')
```

How you'd like to be able to filter events

If the data is really an array, then that's how you'd like to process it. The problem is that you don't actually have the array—not yet anyway. Think about it, though; you don't need to process *any* of the data until the values are needed somewhere else in the program. The manipulations that will be done on the data when it's actually needed can be defined up front, before any of the data is available. Treating a series of asynchronous events as an array is an abstraction—one that makes it easier to compose events in familiar ways.

9.4. Event composition

In this section you'll see how to hide the plumbing of an event emitter to make it easier to compose events. This starts by learning to be lazy.